

Efficient Homomorphic Integer Computer from CKKS

Jaehyung Kim

Stanford University, Stanford, USA

jaehk@stanford.edu

Abstract. As Fully Homomorphic Encryption (FHE) enables computation over encrypted data, it is a natural question of how efficiently it handles standard integer computations like 64-bit arithmetic. It has long been believed that the CGGI/DM family or the BGV/BFV family are the best options, depending on the size of the parallelism. The discrete variant of CKKS, suggested by Drucker *et al.* [J.Cryptol.'24], provides an interesting alternative for integer computations. Notably, the modular reduction framework proposed by Kim and Noh [CiC'25] built on top of the CKKS-style functional bootstrapping by Bae *et al.* [Asiacrypt'24] gives an efficient arithmetic modulo small integers.

In this work, we propose a novel homomorphic computer for unsigned integer computations. We represent a large integer (e.g. 64-bit) as a vector of smaller chunks (e.g. 4-bit) and construct arithmetic operations relying on discrete CKKS. The proposed scheme supports many of the operations supported in TFHE-rs while outperforming it in terms of amortized running time. Notably, our homomorphic 64-bit multiplication takes 8.85ms per slot, which is more than three orders of magnitude faster than TFHE-rs.

Keywords: Fully Homomorphic Encryption · Integer Arithmetic · Discrete CKKS

1 Introduction

Fully Homomorphic Encryption (FHE) is a branch of cryptography that allows computation in an encrypted state. Since Gentry's first instantiation [Gen09], the efficiency of FHE has been improved dramatically. The major FHE schemes can be categorized into LWE-based and RLWE-based depending on whether the default ciphertext format is LWE or RLWE, respectively. LWE-based schemes such as the CGGI/DM family [CGGI16, DM15] are known to be fast (in terms of latency) and flexible, while RLWE-based schemes such as the BGV/BFV family [Bra12, FV12, BGV12] and CKKS [CKKS17] have better throughput. One of the key technical differences is that computations on LWE-based schemes mostly rely on programmable/functional bootstrapping [CJP21, KS23] whereas computations on RLWE-based schemes mainly use homomorphic polynomial evaluations (i.e. addition and multiplication).

It had long been believed that different families are good at different functionalities. For instance, the CGGI/DM family was used to handle small or non-parallelizable computations, the BGV/BFV family was used to handle parallelizable large integer computations, and CKKS was used to handle real number arithmetic. However, recent improvements in the discrete variant of CKKS [DMPS24, CKKL24, BCKS24, BKSS24, AKP25, KN25] suggest that this may not be true. In particular, recent works [BCKS24, BKSS24, AKP25] show that CKKS handles functional/programmable bootstrapping more efficiently than CGGI/DM by several orders of magnitude. As functional/programmable bootstrapping is

a core component of CGGI/DM, this means that it can be preferable to use CKKS if we have sufficient parallelism.

Since we know that CKKS handles homomorphic look-up tables efficiently, a natural question is whether CKKS is good at integer computations in general. For computations modulo NTT-friendly primes, it seems that the BGV/BFV family is the right choice, as they provide a fast and efficient solution for both latency and throughput, in light of recent improvements in BGV/BFV bootstrapping [MHWW24, KSS24] and generalization of BFV [GV25, CHM+25]. However, when it comes to standard modulo power-of-two computations like unsigned 64-bit arithmetic, the landscape is relatively unclear. The most straightforward option, to use the plaintext modulus to be a power-of-two has several problems. One problem is that the plaintext modulus is too large to be efficiently supported. For instance, 64-bit multiplications in BGV/BFV enlarge the noise by more than 64 bits, and choosing large parameters like large ring degree (e.g. $\log(N) \geq 17$) is unavoidable.¹ Recall that large parameters lead to inefficiency in terms of latency and memory footprint, which is not desirable. Another problem is that the use of a power-of-two modulus prevents one from using large parallelism. In BGV/BFV, the number of slots is determined by how the cyclotomic polynomial $\Phi_M(X)$ splits in the plaintext space $\mathbb{Z}_t$. As t is a power-of-two, we do not have as many slots as in the case of NTT-friendly primes, leading to lower throughput for handling modulo t computations in parallel.² In addition, directly using computations over $\mathbb{Z}_t$ makes it difficult to handle important components of integer arithmetic such as bit shift and comparison.

Instead of directly supporting $\mathbb{Z}_t$ arithmetic for large t , the existing approaches often decompose t into several chunks to improve efficiency. For instance, several works [CKK16, XCWF16, QZL+19, ZQH+21, HZY+22] considered radix-2 arithmetic operations for handling integer computations. Similarly, [TLW+21, IZ21] decompose large modulus into smaller finite field elements, leading to efficient homomorphic comparison. By decomposing the desired plaintext modulus t , one can keep the noise growth small while supporting efficient homomorphic operations like comparison and bit shifts. However, as modulo 2 (or 2^ℓ for small ℓ) plaintext space still has a limited number of slots, the SIMD capability or throughput is not as great as the case of NTT-friendly plaintext modulus.

An alternative option is to use the other families, such as CGGI/DM or CKKS. Using CGGI/DM to handle integer computations is relatively well explored, and the major libraries like TFHE-rs [Zam22] allow standard power-of-two modulus integer computations by using lower precision arithmetic as a building block. On the other hand, using CKKS to handle integer computations is new (motivated in [DMPS24] and improved in [BCKS24, BKSS24, AKP25, KN25]) and has not yet been fully explored.³ For small precision like 4 or 8 bits, one can regard integers as real numbers and rely on CKKS operations, look-up tables [CKKL24], and modular reduction [KN25].⁴ For large precision, a straightforward approach is to use high-precision CKKS and directly support $\mathbb{Z}_t$ for large power-of-two integer t , but this has similar problems as in BGV/BFV, such as large parameters and lack of efficient comparison/bit shift.

In this regard, it can be tempting to use a decomposition-based approach as in

¹The standard BGV/BFV bootstrapping (e.g. [MHWW24, GIKV23]) is a polynomial evaluation over a large modulus, leading to huge modulus consumption. For instance, the bootstrapping in [GIKV23] for 2^{51} modulus consumes ≈ 1000 bits for digit removal. Instead, one could use the BFV bootstrapping from CKKS [KSS24], but it needs many META-BTS [BCC+22] iterations.

²If M is odd, $\Phi_M(X)$ splits into $\phi(M)/\text{ord}_M(2)$ irreducible factors, which corresponds to the number of slots [GV23, Section 2.3]. Hence there is at most $\phi(M)/\log_2(M)$ slots ($\because \text{ord}_M(2) \geq \log_2(M)$). Concretely, [GIKV23] uses $M = 42799$ and $\varphi(42799)/21 = 2016$ slots. For a similar ring dimension $\varphi(M) = 32768$, CKKS has $\varphi(M) = 32768$ real slots, which is more than an order of magnitude more.

³For SIMD functional bootstrapping, one may consider other approaches (e.g. [MS18, LW23, PMH+25]), but we consider the CKKS-based approach for efficiency.

⁴A concurrent work [NHY+25] focuses on avoiding XOR operations by relying on RLWE modular reduction (as in [KN25]).

BGV/BFV. Indeed, [YZ⁺24] suggests using decomposition to efficiently handle both homomorphic multiplication and comparison. However, their arithmetic operations do not modular reduce, and individual digits exceed the base and continuously grow. As a result, their approach cannot support bootstrapping and can evaluate only circuits of predetermined size. To support bootstrapping, we need to take care of the carries every time we perform homomorphic arithmetic operations. Since CKKS does not naturally support modular reduction, we need a different strategy than the ones in the BGV/BFV family. In this paper, we suggest using the modular reduction in [KN25] to efficiently instantiate fully homomorphic encryption over large unsigned integers (i.e. arithmetic over $\mathbb{Z}_{2^k}$ for large k).

1.1 Technical Overview

We provide a simplified overview of our method, focusing on how the underlying message behaves through homomorphic operations. At a high level, we decompose a large integer into digits and rely on the textbook addition/multiplication of a convolution form that computes iteratively from the lowest digit. Let d be the base for our digit decomposition, and let $t = d^u$ be the target modulus. Our goal is to enable $\mathbb{Z}_t$ arithmetic using digit decomposition of base d .

Ingredients

We rely on a homomorphic computing system that supports addition and multiplication over $\mathbb{Z}$ (up to a moderate precision) and modular reduction $[\cdot]_d$ for some $d \in \mathbb{Z}_{>0}$.⁵ Such a homomorphic computer can be instantiated via the discrete variant of CKKS (based on [DMPS24, KN25]), which we discuss in detail in Section 2. The computing system has a particular efficiency property that modular reduction is much more costly than addition and multiplication. Due to this property, we need arithmetic algorithms that minimize the number of modular reductions.

Reduction

Let $\sum_{i=0}^{u-1} a_i \cdot d^i$ be a representation of a modulo $\mathbb{Z}_{d^u}$ integer, where each a_i could possibly be greater than or equal to d . The reduction operation converts it to a reduced form so that all the digits belong to $\{0, 1, \dots, d-1\}$ as in the standard representation. To minimize the number of modular reductions, we can iteratively compute the i th digit starting from the least significant digit. To be precise, the iterations work as follows.

- At step 0, we divide a_0 by d and get quotient and remainder q_0 and r_0 , respectively. We then set $c_1 := q_0 + a_1$.
- At step $1 \leq i < u-1$, we divide c_i by d and get quotient and remainder q_i and r_i , respectively. We then set $c_{i+1} = q_i + a_{i+1}$.
- At step $u-1$, we compute $r_{u-1} = [c_{u-1}]_d$ and output $\sum_{i=0}^{u-1} r_i \cdot d^i$.

This leads to u modular reductions in total.

Difficulty

Although the definition of reduction is straightforward, there is a hidden obstacle in the construction. That is, division by d consumes ciphertext modulus at least by d , leading to a continuous decline in modulus. In particular, the modulus of c_i is at least d^i times smaller than that of a_0 , significantly increasing the overall FHE parameters. To tackle this

⁵The choice of d affects the performance of the modular reduction. A typical choice is 4-8 bits.

problem, we bootstrap some of the c_i 's using iterative discrete bootstrapping from [KN25]. The number of bootstrapping iterations to bootstrap c_i depends on the size of c_i : we may use β discrete bootstrappings for $c_i \leq d^\beta$. It is important to keep the size of each a_i small during homomorphic operations so that each c_i becomes small.

Multiplication

We describe our multiplication algorithm, which can be written as a combination of convolution and reduction. Let $a = \sum_{i=0}^{u-1} a_i \cdot d^i$ and $b = \sum_{i=0}^{u-1} b_i \cdot d^i$ be digit decompositions of two integers $a, b \in \mathbb{Z}_{d^u}$. We observe that

$$[ab]_{d^u} = \sum_{i=0}^{u-1} \left(\sum_{j=0}^i a_j b_{i-j} \right) \cdot d^i.$$

Hence, to minimize the number of modular reductions, we first compute a possibly overflowing representation $\left(\sum_{j=0}^i a_j b_{i-j} \right)_{0 \leq i < d}$ and reduce it afterwards. To count the number of bootstrappings needed in total, we measure the size of each $\sum_{j=0}^i a_j b_{i-j}$. We choose $u \leq d$ so that $\sum_{j=0}^i a_j b_{i-j} \leq u \cdot d^2 \leq d^3$, resulting in $\approx 3u$ bootstrappings.⁶

Other Operations

Our idea naturally extends to other operations, including addition, subtraction, comparison, and bit shift. Addition and subtraction work similarly to multiplication: we first perform digit-wise addition/subtraction and reduce afterward. Interestingly, a homomorphic comparison $a \geq b$ between two u -digit integers a and b can be interpreted as a special case of subtraction, as we can first compute $a - b$ as $u + 1$ -digit subtraction and extract the highest digit. If the highest digit is 0, then it means that $a \geq b$, and otherwise the highest digit is -1 . For a homomorphic shift operation by a scalar, we shift the digits and apply modular reductions. For instance, if we left shift by $y < \log_2(d)$, then the i th digit can be written as

$$lo(a_i \ll y) + hi(a_{i-1} \ll y)$$

where $(a_0, a_1, \dots, a_{u-1})$ is the input and lo and hi refers to remainder and quotient modulo d , respectively.

As a result, our homomorphic computer is equipped with frequently used unsigned integer operations with reasonable performance, serving as a perfect baseline for implementing any applications based on integer operations.

1.2 Contribution

We propose an efficient Single Instruction Multiple Data (SIMD) FHE over integers that supports a wide range of operations. In particular, our scheme provides three different types of operations, namely arithmetic operations, comparison, and bootstrapping. Note that these operations are the most important primitives of FHE and are necessary for most applications.

Concrete Efficiency

Among the schemes that support diverse operations, our scheme provides the best throughput. As we rely on CKKS, we enjoy almost the maximum parallelism possible (i.e. $N/2$

⁶ $O(u^2)$ homomorphic multiplications are negligible compared to the $O(u)$ modular reductions as long as u is not too large.

slots), outperforming the approaches based on other FHE schemes. We compare the concrete performance on homomorphic multiplication and comparison in Table 1 and 2. Compared to the widely used TFHE-rs [Zam22], our multiplication and comparison are roughly three and two orders of magnitude faster, respectively. When compared with radix-based approaches using BGV/BFV, our multiplication and comparison are approximately two and one orders of magnitude faster, respectively.

Table 1: Homomorphic multiplication over $\mathbb{Z}_{2^k}$.

	k	λ	amortized time (ms)
[HZY ⁺ 22]	32	80	1020
[Zam22]	32	128	6420
	64		25400
Ours	32	≈ 128	4.25
	64		8.85

Table 2: Homomorphic comparison over $\mathbb{Z}_{2^{64}}$.

	λ	# slots	latency (sec)	amortized time (ms)
[TLW ⁺ 21]	> 80	256	20.2	78.8
		128	20.5	160
		16	4.75	297
[Zam22]	128	1	0.685	685
Ours	≈ 128	16384	102	6.23

Compared to the possible approaches that directly encode large integers without decomposition, our method achieves much smaller FHE parameters. As arithmetic over $\mathbb{Z}_t$ for large t like 64-bit consumes a huge amount of modulus per multiplication, it is unavoidable for the direct encoding approaches to use large ring dimension such as $\log N = 17, 18$. On the other hand, we use an FHE parameter for $\mathbb{Z}_d$ to instantiate modulo at most d^d arithmetic, having significantly smaller parameters to achieve the same precision. Indeed, we were able to instantiate 64-bit precision arithmetic using $\log N = 15$ parameters with only ≈ 800 bits of modulus budget.

Versatility

Our scheme is not only efficient but also offers a variety of operations. Traditionally, SIMD schemes were only capable of computing addition and multiplication, leading to inefficiency in computing other types of functions. On the contrary, our scheme supports arithmetic, comparison, and shift operations, leading to a wider set of instructions to emulate the (unencrypted) computer. Furthermore, our scheme is compatible with the multi-precision arbitrary function evaluation in [AKP25], supporting any functions over unsigned integers. In addition, our method gives CKKS the ability to efficiently compute over large integers, which means that the capability of CKKS is extended to both real numbers and integers. As our encoding is directly compatible with the CKKS encoding, one can efficiently convert from our encoding to CKKS and vice versa, to maximize efficiency.

Enhanced Security

As the original CKKS cannot distinguish the error from the message, it was difficult for CKKS to achieve advanced security notions such as IND-CPA^D security [LM21] or Threshold FHE security [AJL⁺12, BGG⁺18]. Since the discrete CKKS framework allows us to distinguish the error from the ciphertext by discretizing the message space, it can

enjoy the advantages of the exact schemes in terms of security. As we provide an efficient integer computer, discrete CKKS supports any operation without going through the original CKKS, leading to better security. For instance, one may use our homomorphic fixed point arithmetic instead of the usual CKKS operations to avoid large noise flooding needed to achieve advanced security.

2 Preliminaries

Let $N > 1$ be a power of two integer and $Q > 1$ be an integer. Let $\mathcal{R} = \mathbb{Z}[X]/(X^N + 1)$ and $\mathcal{R}_Q = \mathcal{R}/Q\mathcal{R}$. Let $\text{DFT} : \mathbb{R}[X]/(X^N + 1) \rightarrow \mathbb{C}^{N/2}$ be a discrete Fourier transform (DFT) defined as $m(X) \mapsto \left(m(\zeta^{5^i})\right)_{0 \leq i < N/2}$ where ζ is a complex primitive $2N$ -th root of unity. Let $\text{iDFT} : \mathbb{C}^{N/2} \rightarrow \mathbb{R}[X]/(X^N + 1)$ be its inverse.

2.1 CKKS Basics

In CKKS, there are two kinds of encoding called slots-encoding and coeffs-encoding. The **slots-encoding** $\text{Ecd} : \mathbb{C}^{N/2} \rightarrow \mathcal{R}$ is defined as

$$\text{Ecd}(\vec{z}) = \lfloor \Delta \cdot \text{iDFT}(\vec{z}) \rfloor$$

where $\Delta > 0$ is a scaling factor. The slots-decoding $\text{Dcd} : \mathcal{R} \rightarrow \mathbb{C}^{N/2}$ is defined as

$$\text{Dcd}(m(X)) = \frac{1}{\Delta} \cdot \text{DFT}(m(X)).$$

The slots-encoding supports SIMD computations and is used as a default encoding for CKKS. The **coeffs-encoding** $\text{CoeffEcd} : \mathbb{R}^N \rightarrow \mathcal{R}$ is defined as

$$\text{CoeffEcd}(\vec{z}) = \sum_{i=0}^{N-1} \lfloor \Delta \cdot z_i \rfloor \cdot X^i$$

where $\vec{z} = (z_0, z_1, \dots, z_{N-1})$, simply scaling up the vector and round. The coeffs-decoding $\text{CoeffDcd} : \mathcal{R} \rightarrow \mathbb{R}^N$ is its approximate inverse defined as

$$\text{CoeffDcd} \left(\sum_{i=0}^{N-1} m_i X^i \right) = \frac{1}{\Delta} (m_0, m_1, \dots, m_{N-1}).$$

As coeffs-encoding is compatible with coefficient-wise operations, we use it for bootstrapping (to raise modulus), conversions to/from other schemes [BGGJ20], or modular reduction [KN25].

In CKKS, each homomorphic multiplication increases the scaling factor from Δ to Δ^2 , and we rescale the ciphertext to keep the scaling factor to be $\simeq \Delta$. As rescaling reduces the ciphertext modulus, the ciphertext modulus gradually decreases and cannot allow further multiplications at some point. The **CKKS bootstrapping** [CHK⁺18] increases the modulus, recovering the multiplicative capability.

Definition 1 (CKKS Bootstrapping). Let $q, Q > 1$ be integers such that $Q > q$. Let $\text{ct} \in \mathcal{R}_q^2$ be a CKKS ciphertext encrypting a vector $\vec{z} \in \mathbb{C}^{N/2}$ via the slots-encoding, where both real and imaginary parts of each entry are in $[-1, 1]$. The CKKS bootstrapping $\text{BTS} : \mathcal{R}_q^2 \rightarrow \mathcal{R}_Q^2$ raises the modulus while approximately preserving the underlying message. That is,

$$\text{BTS}(\text{ct}) = \text{ct}' \in \mathcal{R}_Q^2$$

where $\text{Dcd} \circ \text{Dec}(\text{ct}) \simeq \text{Dcd} \circ \text{Dec}(\text{ct}')$. Here Dec denotes the CKKS decryption, i.e., taking an inner product with a secret key $\text{sk} = (1, s) \in \mathcal{R}^2$.

The standard CKKS bootstrapping [CHK⁺18] can be simplified as a combination of modulus raising (denoted as **ModRaise**) and modular reduction (denoted as **EvalMod**). If one embeds a CKKS ciphertext $\text{ct} \in \mathcal{R}_{q_0}^2$ at the lowest modulus q_0 into a larger modulus Q , a small integral error term I is added, and the underlying plaintext polynomial m becomes $m + q_0 \cdot I$. The **EvalMod** procedure homomorphically evaluates a modular reduction to remove $q_0 \cdot I$.

An additional issue is that **ModRaise** and **EvalMod** are compatible with different encoding structures, coeffs-encoding and slots-encoding, respectively. Hence, we need to convert slots encoding to coeffs encoding and vice versa, denoted as **StC** and **CtS**, respectively. As a result, the CKKS bootstrapping is instantiated with the order of **StC-ModRaise-CtS-EvalMod**, as illustrated in Algorithm 1.

Algorithm 1: Slot Bootstrapping [BCC⁺22]

Setting : $\Delta \ll q_0$.

Input : $\text{ct} = \text{Enc} \circ \text{Ecd}(\vec{z}) \in \mathcal{R}_q^2$ with $\vec{z} \in [-1, 1]^{N/2}$.

Output : $\text{ct}_{\text{out}} \in \mathcal{R}_Q^2 = \text{Enc} \circ \text{Ecd}(\vec{w})$, where $\vec{w} \simeq \vec{z}$.

- 1 $\text{ct}_{\text{out}} \leftarrow \text{EvalMod} \circ \text{CtS} \circ \text{ModRaise} \circ \text{StC}(\text{ct});$
 - 2 **return** ct_{out} .
-

2.2 Discrete CKKS

The recent improvements in the discrete variant of CKKS (first formalized in [DMPS24]) use a subspace of $\mathbb{C}$ to handle discrete data. For instance, one may restrict the message space to $\mathbb{Z} \subset \mathbb{C}$. We observe that the inclusion $\mathbb{Z} \hookrightarrow \mathbb{C}$ is a ring homomorphism, and addition and multiplication over integers are directly inherited from CKKS. Importantly, this framework allows us to distinguish the underlying error from the ciphertext, which was not possible in the original CKKS.

Definition 2 (Discrete CKKS Ciphertext). Let $U \subseteq \mathbb{C}$ be a finite set. A discrete CKKS ciphertext encrypting a vector $\vec{z} \in U^{N/2}$ is a CKKS ciphertext ct that decrypts to $\vec{z} + \vec{e} \in \mathbb{C}^{N/2}$ where $\vec{e}$ is small. In this case, we call $\vec{z}$ and $\vec{e}$ as the underlying message and error of the discrete CKKS ciphertext ct , respectively.

As the error can be identified, we may reduce the error by evaluating a cleaning polynomial. For instance, [DMPS24] describes a cleaning function $h_1(x) = 3x^2 - 2x^3$ for the encoding set $U = \{0, 1\} \subset \mathbb{C}$. The function h_1 satisfies $h_1(0) = 0$ and $h_1(1) = 1$, and has vanishing derivatives on $\{0, 1\}$, thereby sending points close to 0 and 1 closer to 0 and 1, respectively.

Definition 3 (Cleaning). Let $U \subseteq \mathbb{C}$ be a finite set. Let $\text{ct} \in \mathcal{R}_q^2$ be a discrete CKKS ciphertext encrypting a vector $\vec{z} \in U^{N/2}$ whose underlying error is $\vec{e} \in \mathbb{C}^{N/2}$. The cleaning function maps ct to ct' , so that the underlying error of ct' , denoted as $\vec{e}'$ is much smaller than $\vec{e}$.

Discrete CKKS can efficiently handle two types of operations, namely arithmetic operations and interpolation. Arithmetic operation refers to addition and multiplication over $\mathbb{Z}$ inherited from CKKS, whereas interpolation refers to polynomial interpolations over a finite number of points on the complex plane. For interpolation, [CKKL24] suggests using complex roots-of-unity for polynomial interpolations, as it provides more numerically stable interpolation than the typical equispaced points on the real line (e.g. $\{0, 1, \dots, d-1\} \subseteq \mathbb{C}$). The power of interpolation is that one can evaluate an arbitrary function $\mathbb{Z}_d \rightarrow \mathbb{C}$ for some small integer d . Note that the original CKKS needs to rely on polynomial approximations and thus could not compute discontinuous functions efficiently.

Definition 4 (Interpolation over roots-of-unity). Let $U_d = \{1, \omega, \dots, \omega^{d-1}\} \subseteq \mathbb{C}$ where ω is a primitive d -th root of unity. Let $f : \mathbb{Z}_d \rightarrow \mathbb{C}$ be an arbitrary function. The homomorphic look-up table $\text{LUT}_f : \mathcal{R}_Q^2 \rightarrow \mathcal{R}_Q^2$ homomorphically evaluates an interpolation that corresponds to f . That is, $\omega^\alpha \in U_d$ is mapped to $f(\alpha)$ for each slot.

[BCKS24, BKSS24, AKP25] build an analogue of functional/programmable bootstrapping in CGGI/DM [CJP21, KS23] for discrete CKKS. In other words, they design new bootstrapping circuits dedicated to discrete data that evaluate an arbitrary function during bootstrapping. Notably, the CKKS-style functional bootstrapping (i.e. discrete bootstrapping) can be regarded as a parallelization of CGGI/DM bootstrapping which is several orders of magnitude faster than the state-of-the-art such as [Zam22]. We provide a (simplified) instantiation of discrete bootstrapping from [BKSS24] in Algorithm 2. Here EvalExp denotes a homomorphic evaluation of the complex exponential function $x \mapsto e^{2\pi i x}$.

Algorithm 2: Discrete Bootstrapping [BKSS24]

Setting : $U = \{0, 1, \dots, d-1\} \subset \mathbb{C}$.
Input : $\text{ct} \in \mathcal{R}_Q^2$ a discrete CKKS ciphertext encrypting $\vec{z} \in U^{N/2}$. $f : \mathbb{Z}_d \rightarrow \mathbb{C}$ an arbitrary function.
Output : ct_{out} encrypting $f(\vec{z})$.
1 $\text{ct}_{\text{out}} \leftarrow \text{LUT}_f \circ \text{EvalExp} \circ \text{CtS} \circ \text{ModRaise} \circ \text{StC}(\text{ct})$;
2 **return** ct_{out} .

More recently, [KN25] suggested using discrete bootstrapping to instantiate homomorphic modular reduction. Recall that the original CKKS does not have an inherent modular reduction, unlike other schemes like BGV/BFV. The main reason is that CKKS encodes messages in the least significant bits via DFT, which is not compatible with modular reduction. However, [KN25] observed that we can indeed use the native modular reduction at the bottom modulus by incorporating discrete bootstrapping as a subroutine. For instance, simply choosing f as an identity function and inserting an RLWE modular reduction between ModRaise and StC gives a modular reduction by a smaller integer d denoted as IntMod_d . See Algorithm 3 for details.

Algorithm 3: IntMod_d [KN25]

Setting : $\text{id} : \mathbb{Z}_d \rightarrow \mathbb{Z}_d$ an identity map, and $\Delta_0 = q_0/d$.
Input : $\text{ct} \in \mathcal{R}_Q^2$ a discrete CKKS ciphertext encrypting $\vec{z} \in \mathbb{Z}^{N/2}$.
Output : ct_{out} encrypting $[\vec{z}]_d$.
1 $\text{ct}_{\text{out}} \leftarrow \text{LUT}_{\text{id}} \circ \text{EvalExp} \circ \text{CtS} \circ \text{ModRaise} \circ [\text{StC}(\text{ct})]_{q_0}$;
2 **return** ct_{out} .

As a summary of the previous works, we provide a simplified overview of discrete CKKS in Figure 1. In discrete CKKS, there are three separated ciphertext formats providing different operations. Here, separated means that one can only perform one type of operation in a single format and needs to evaluate some (expensive) transformations to convert it to the other formats. To be specific, three formats can be denoted as arithmetic, coefficient, and interpolation formats. First, the arithmetic format refers to the usual CKKS slots-encoding, which supports addition and multiplication inherited from the original CKKS. In terms of discrete computations, one can use this type for integer additions and multiplications. Second, the coefficients format refers to the coeffs-encoded CKKS ciphertext at the bottom modulus. This type of ciphertexts are used to perform RLWE modular reduction as suggested in [KN25] or conversions between RLWE/MLWE/LWE (one may refer to [BCK+23]). Third, the interpolation format refers

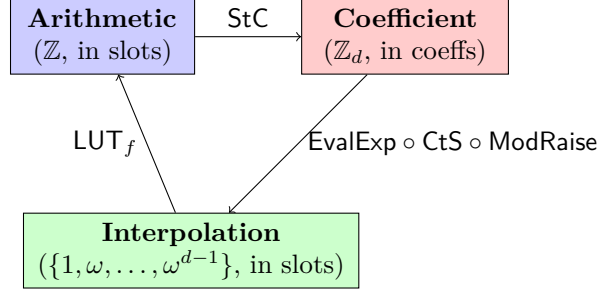


Figure 1: Overview of Discrete CKKS

to discrete CKKS ciphertexts encoded via roots-of-unity encoding as in [CKKL24], suitable for handling polynomial interpolations. It would usually be integrated with bootstrapping as in [BKSS24, AKP25], but one can specifically use this kind of encoding to handle arbitrary functions more efficiently. The arith-to-coeff, coeff-to-interpolate, interpolate-to-arith conversions can be instantiated with StC , $\text{EvalExp} \circ \text{CtS} \circ \text{ModRaise}$, and LUT_f , respectively.

3 Proposed Method

In this section, we design a homomorphic computer that supports standard unsigned integer computations (e.g. 64-bit arithmetic), on top of the discrete CKKS framework introduced in the previous section. Let $t = 2^k$ be the target modulus for which we want to evaluate. As a high-level overview, we decompose t via a digit decomposition by a smaller modulus $d = 2^\ell$ and handle modulo t computation using several modulo d computations. For instance, we may choose $k = 64$ and $\ell = 4$, leading to evaluating 64-bit computation using $k/\ell = 16$ digits of modulo $2^\ell = 16$ computations.

3.1 Scheme Description

We first illustrate how we put k -bit data into discrete CKKS ciphertexts. The message in our encoding is a vector of k -bit integers $\vec{z} = (z_0, z_1, \dots, z_{N/2-1}) \in \mathbb{Z}_t^{N/2}$. To encrypt this vector, we use $u = k/\ell$ ciphertexts $\text{ct}_0, \text{ct}_1, \dots, \text{ct}_{u-1}$ where ct_i encrypts the i th digit of the base- d representation of $\vec{z}$. That is,

$$\text{ct}_i = \text{Enc} \circ \text{Ecd}(z_{0i}, z_{1i}, \dots, z_{(N/2-1)i})$$

where $z_j = \overline{z_{j(u-1)} z_{j(u-2)} \cdots z_{j0}} (d)$ for each $0 \leq j < N/2$. Here, each ciphertext ct_i for $0 \leq i < u$ is a valid discrete CKKS ciphertext for $\mathbb{Z}_d$ computations. Therefore, we may use any operations in the prior works of discrete CKKS, such as arithmetic operations, table look-ups, and modular reduction. In particular, we mainly rely on addition, multiplication, and modular reduction to handle ring operations over $\mathbb{Z}_d$ as well as the carry operation. To keep the discrete bootstrapping for $\mathbb{Z}_d$ efficient, we choose d to be sufficiently small (e.g. $d \leq 2^8$). In terms of cleaning, we may use digit-wise cleaning for $\mathbb{Z}_d$ (e.g. cleaning for d -th roots of unity as in [CKKL24]). We elaborate on the encryption scheme as follows.

- **Encryption:** Let $\vec{z} = (z_0, z_1, \dots, z_{N/2-1}) \in \mathbb{Z}_t^{N/2}$ be a vector of k -bit unsigned integers. The encryption $\text{IntEnc} : \mathbb{Z}_t^{N/2} \rightarrow (\mathcal{R}_Q^2)^u$ is defined as

$$\text{IntEnc}(\vec{z}) = (\text{Enc} \circ \text{Ecd}(z_{0i}, z_{1i}, \dots, z_{(N/2-1)i}))_{0 \leq i < u}$$

where $z_j = \overline{z_{j(u-1)} z_{j(u-2)} \cdots z_{j0}} (d)$ is the base d representation of z_j for each $0 \leq j < N/2$.

- **Decryption:** Let $\text{ct} = (\text{ct}_0, \text{ct}_1, \dots, \text{ct}_{u-1})$ be a vector of discrete CKKS ciphertexts. The decryption $\text{IntDec} : (\mathcal{R}_Q^2)^u \rightarrow \mathbb{Z}_t^{N/2}$ is defined as

$$\text{IntDec}(\text{ct}) = \sum_{i=0}^{u-1} [\text{Dcd} \circ \text{Dec}(\text{ct}_i)] \cdot d^i.$$

- **Bootstrapping:** Given a ciphertext $\text{ct} = (\text{ct}_0, \text{ct}_1, \dots, \text{ct}_{u-1})$ encrypting a vector $\vec{z} \in \mathbb{Z}_t^{N/2}$, the bootstrapping $\text{Boot} : (\mathcal{R}_Q^2)^u \rightarrow (\mathcal{R}_Q^2)^u$ is defined as

$$\text{Boot}(\text{ct}) = (\text{IntBoot}_d(\text{ct}_i))_{0 \leq i < u}$$

where $\text{IntBoot}_d : \mathcal{R}_Q^2 \rightarrow \mathcal{R}_Q^2$ refers to an identity discrete bootstrapping (i.e. functional bootstrapping evaluating an identity function) for $\mathbb{Z}_d$. For simplicity, we assume that this bootstrapping not only raises the modulus but also reduces the noise. See [BKSS24, AKP25] for details.

Theorem 1 (Encryption Correctness). *Let $\vec{z} = (z_0, z_1, \dots, z_{N/2-1}) \in \mathbb{Z}_t^u$ be a vector of k -bit unsigned integers. Then we have*

$$\text{IntDec} \circ \text{IntEnc}(\vec{z}) = \vec{z}.$$

Proof. Observe that $z_j = \sum_{i=0}^{u-1} z_{ji} \cdot d^i$ from the base- d representation of z_j . By the correctness of CKKS encryption and decryption, we have

$$\begin{aligned} \text{IntDec} \circ \text{IntEnc}(\vec{z}) &= \sum_{i=0}^{u-1} (z_{0i}, z_{1i}, \dots, z_{(N/2-1)i}) \cdot d^i \\ &= \left(\sum_{i=0}^{u-1} z_{ji} \cdot d^i \right)_{0 \leq j < N/2} = (z_0, z_1, \dots, z_{N/2-1}) = \vec{z}. \end{aligned}$$

□

Theorem 2 (Bootstrapping Correctness). *Let $\text{ct} \in (\mathcal{R}_Q^2)^u$ be a ciphertext encrypting a vector $\vec{z} \in \mathbb{Z}_t^{N/2}$ according to the above encryption. Then we have*

$$\text{IntDec} \circ \text{Boot}(\text{ct}) = \vec{z}.$$

Proof. Let $\text{ct} = (\text{ct}_0, \text{ct}_1, \dots, \text{ct}_{u-1})$ and $\vec{z} = (z_0, z_1, \dots, z_{N/2-1})$. Let $z_j = \overline{z_{j(u-1)} z_{j(u-2)} \cdots z_{j0}}$ (d) be the base- d representation of z_j for each $0 \leq j < N/2$. Let $\vec{w}_i = (z_{0i}, z_{1i}, \dots, z_{(N/2-1)i}) \in \mathbb{Z}_d^{N/2}$ be a vector of ℓ -bit unsigned integers for each $0 \leq i < u$. Then by the definition of the encryption, we have that ct_i is a valid discrete CKKS encryption of $\vec{w}_i$ for each i . Recall that the bootstrapping for ct is defined as an element-wise integer bootstrapping, which means that the components of $\text{Boot}(\text{ct})$ are the bootstrappings of ct_i . Therefore, we have

$$\text{IntDec} \circ \text{Boot}(\text{ct}) = \sum_{i=0}^{u-1} [\text{Dcd} \circ \text{Dec} \circ \text{Boot}(\text{ct}_i)] \cdot d^i = \sum_{i=0}^{u-1} [\text{Dcd} \circ \text{Dec}(\text{ct}_i)] \cdot d^i = \vec{z}.$$

The second equality follows from the correctness of IntBoot_d . □

3.2 Arithmetic Operations

Based on the encryption scheme defined in the previous subsection, we describe how we define homomorphic arithmetic operations over $\mathbb{Z}_t$. We not only provide simple addition and multiplication but also some popular operations (e.g. comparison, bit shift) that are defined over unsigned integers. Recall that IntMod_d is a homomorphic modular reduction by $\mathbb{Z}_d$ as defined in Algorithm 3 and $\text{Carry}_d = (\text{id} - \text{IntMod}_d)/d$.⁷

- **Reduction:** Let $\text{ct} = (\text{ct}_0, \text{ct}_1, \dots, \text{ct}_{u-1}) \in (\mathcal{R}_Q^2)^u$ be a vector of discrete CKKS ciphertexts encrypting vectors in $\mathbb{Z}^{N/2}$. The modular reduction of ct is sequentially defined as

$$\text{Reduce}(\text{ct}) = (\text{ct}'_0, \text{ct}'_1, \dots, \text{ct}'_{u-1})$$

where $\text{ct}'_0 = \text{IntMod}_d(\text{ct}_0)$, $\text{ct}''_0 = \text{Carry}_d(\text{ct}_0)$, and

$$\text{ct}'_i = \text{IntMod}_d(\text{ct}''_{i-1} + \text{ct}_i), \quad \text{ct}''_i = \text{Carry}_d(\text{ct}''_{i-1} + \text{ct}_i)$$

for each $1 \leq i < u$. The result of Reduce encrypts a message

$$\sum_{i=0}^{u-1} [\text{Dcd} \circ \text{Dec}(\text{ct}_i)] \cdot d^i$$

modulo t , i.e., reducing the ciphertext ct to the correct base- d representation. Let $\text{Carry}(\text{ct})$ be defined as ct''_{u-1} .

- **Addition:** Let $\text{ct} = (\text{ct}_i)_{0 \leq i < u}$, $\text{ct}' = (\text{ct}'_i)_{0 \leq i < u} \in (\mathcal{R}_Q^2)^u$ be vectors of ciphertexts each encrypting a k -bit unsigned integer according to the encryption scheme in Section 3.1. The addition of ct and ct' is defined as

$$\text{Add}(\text{ct}, \text{ct}') = \text{Reduce}((\text{ct}_i + \text{ct}'_i)_{0 \leq i < u})$$

where the addition on the right-hand side is the usual CKKS addition.

- **Subtraction:** Let $\text{ct} = (\text{ct}_i)_{0 \leq i < u}$, $\text{ct}' = (\text{ct}'_i)_{0 \leq i < u} \in (\mathcal{R}_Q^2)^u$ be vectors of ciphertexts each encrypting a k -bit unsigned integer according to the encryption scheme in Section 3.1. The subtraction of ct and ct' is defined as

$$\text{Sub}(\text{ct}, \text{ct}') = \text{Reduce}((\text{ct}_i - \text{ct}'_i)_{0 \leq i < u})$$

where subtraction on the right-hand side is the usual CKKS subtraction.

- **Multiplication:** Let $\text{ct} = (\text{ct}_i)_{0 \leq i < u}$, $\text{ct}' = (\text{ct}'_i)_{0 \leq i < u} \in (\mathcal{R}_Q^2)^u$ be vectors of ciphertexts each encrypting a k -bit unsigned integer according to the encryption scheme in Section 3.1. The multiplication of ct and ct' is defined as

$$\text{Mult}(\text{ct}, \text{ct}') = \text{Reduce} \left(\left(\sum_{j=0}^i \text{Mult}(\text{ct}_j, \text{ct}'_{i-j}) \right)_{0 \leq i < u} \right)$$

where Mult on the right-hand side refers to the usual CKKS multiplication.

- **Comparison:** Let $\text{ct} = (\text{ct}_i)_{0 \leq i < u}$, $\text{ct}' = (\text{ct}'_i)_{0 \leq i < u} \in (\mathcal{R}_Q^2)^u$ be vectors of ciphertexts each encrypting a k -bit unsigned integer according to the encryption scheme in Section 3.1. The comparison $\text{ct} \geq \text{ct}'$ is defined as

$$\text{ct} \geq \text{ct}' = \text{Carry}(\text{ct} - \text{ct}') + 1$$

which outputs 1 for true and 0 for false.

⁷Note that $\text{Carry}_d(\text{ct})$ may be at a lower level than ct . This may lead to modulus consumption and bootstrapping, but we consider the efficiency aspect in the later subsection.

- **Left Shift:** Let $\mathbf{ct} = (\mathbf{ct}_0, \mathbf{ct}_1, \dots, \mathbf{ct}_{u-1}) \in (\mathcal{R}_Q^2)^u$ be a vector of ciphertexts encrypting a k -bit unsigned integer according to the encryption scheme in Section 3.1. Let $0 \leq m < k$ be an integer. The left shift of $\mathbf{ct}$ by m denoted as $\mathbf{ct} \ll m$ is defined as

$$\mathbf{ct} \ll m = (\text{IntMod}_d(\mathbf{ct}_{i-x} \ll y) + \text{Carry}_d(\mathbf{ct}_{i-x-1} \ll y))_{0 \leq i < u}$$

where $x = \lfloor m/\ell \rfloor$, $y = [m]_\ell$, and $\mathbf{ct}_i = 0$ for $i < 0$.

- **Right Shift:** Let $\mathbf{ct} = (\mathbf{ct}_0, \mathbf{ct}_1, \dots, \mathbf{ct}_{u-1}) \in (\mathcal{R}_Q^2)^u$ be a vector of ciphertexts encrypting a k -bit unsigned integer according to the encryption scheme in Section 3.1. Let $0 \leq m < k$ be an integer. The right shift of $\mathbf{ct}$ by m denoted as $\mathbf{ct} \gg m$ is defined as

$$\mathbf{ct} \gg m = (\text{Carry}_d(\mathbf{ct}_{i+x} \ll (\ell - y)) + \text{IntMod}_d(\mathbf{ct}_{i+x+1} \ll (\ell - y)))_{0 \leq i < u}$$

where $x = \lfloor m/\ell \rfloor$, $y = [m]_\ell$, and $\mathbf{ct}_i = 0$ for $i \geq u$.

Theorem 3 (Arithmetic Correctness). *Reduction, Addition, Subtraction, Multiplication, Comparison, Right Shift, and Left Shift, defined above, are correct.*

Proof. We give proof for each operation.

- Reduction: It suffices to show that

$$\sum_{i=0}^{u-1} [\text{Dcd} \circ \text{Dec}(\mathbf{ct}'_i)] \cdot d^i \equiv_t \sum_{i=0}^{u-1} [\text{Dcd} \circ \text{Dec}(\mathbf{ct}_i)] \cdot d^i.$$

By mathematical induction, we first check that

$$[\text{Dcd} \circ \text{Dec}(\mathbf{ct}''_j)] = \left\lfloor \frac{\sum_{i=0}^j [\text{Dcd} \circ \text{Dec}(\mathbf{ct}_i)] \cdot d^i}{d^j} \right\rfloor$$

holds for each j . Again, we use mathematical induction and get

$$\sum_{i=0}^j [\text{Dcd} \circ \text{Dec}(\mathbf{ct}'_i)] \cdot d^i = \left\lfloor \sum_{i=0}^j [\text{Dcd} \circ \text{Dec}(\mathbf{ct}_i)] \cdot d^i \right\rfloor_{d^{j+1}}$$

for each j . By plugging in $j = u - 1$, we finish the proof.

- Addition/Subtraction: We first check that

$$\sum_{i=0}^{u-1} [\text{Dcd} \circ \text{Dec}(\mathbf{ct}_i \pm \mathbf{ct}'_i)] \cdot d^i = \sum_{i=0}^{u-1} [\text{Dcd} \circ \text{Dec}(\mathbf{ct}_i)] \cdot d^i \pm \sum_{i=0}^{u-1} [\text{Dcd} \circ \text{Dec}(\mathbf{ct}'_i)] \cdot d^i$$

from the decryption correctness of $(\mathbf{ct}_i)_i$ and $(\mathbf{ct}'_i)_i$. Then by the correctness of reduction, we have the correctness of addition/subtraction.

- Multiplication: We first check that

$$\begin{aligned} & \sum_{i=0}^{u-1} \left\lfloor \text{Dcd} \circ \text{Dec} \left(\sum_{j=0}^i \text{Mult}(\mathbf{ct}_j, \mathbf{ct}'_{i-j}) \right) \right\rfloor \cdot d^i \\ &= \sum_{i=0}^{u-1} \left(\sum_{j=0}^i [\text{Dcd} \circ \text{Dec}(\mathbf{ct}_j)] \odot [\text{Dcd} \circ \text{Dec}(\mathbf{ct}'_{i-j})] \cdot d^i \right) \\ &\equiv_t \left(\sum_{i=0}^{u-1} [\text{Dcd} \circ \text{Dec}(\mathbf{ct}_i)] \cdot d^i \right) \odot \left(\sum_{i=0}^{u-1} [\text{Dcd} \circ \text{Dec}(\mathbf{ct}'_i)] \cdot d^i \right) \end{aligned}$$

by the decryption correctness. Then, by the correctness of reduction, we have the correctness of multiplication.

- Comparison: We observe that given an integer x , the corresponding carry operation computes $(x - [x]_t)/t$. Let $\text{ct}' - \text{ct}$ encrypts an integer vector $\vec{z} \in \mathbb{Z}^{N/2}$ according to the decryption function in Section 3.1. Since both ct and ct' encrypt unsigned k -bit integers, we have $-t < z_i < t$ for each entry z_i for $\vec{z}$. In particular, the i th entry of ct is greater than or equal to the i th entry of ct' if and only if $z_i \geq 0$. Note that the carry operation outputs 0 when $0 \leq z_i < t$ and outputs -1 when $-t < z_i < 0$. Therefore, we have that $\text{Carry}(\text{ct} - \text{ct}') + 1$ encrypts the right result.
- Left shift: It suffices to check the base- d representation of the left shift of the original message. Let $\alpha = \overline{\alpha_{u-1}\alpha_{u-2}\cdots\alpha_0}$ be the base- d representation of an unsigned k -bit integer α . Let $\beta = \overline{\beta_{u-1}\beta_{u-2}\cdots\beta_0}$ be the base- d representation of $\alpha \ll m$. For each i ,

$$\beta_i = [\alpha_{i-x} \ll y]_d + \left\lfloor \frac{\alpha_{i-x-1} \ll y}{d} \right\rfloor.$$

This proves the correctness.

- Right shift: It suffices to check the base- d representation of the right shift of the original message. Let $\alpha = \overline{\alpha_{u-1}\alpha_{u-2}\cdots\alpha_0}$ be the base- d representation of an unsigned k -bit integer α . Let $\beta = \overline{\beta_{u-1}\beta_{u-2}\cdots\beta_0}$ be the base- d representation of $\alpha \gg m$. For each i ,

$$\beta_i = [\alpha_{i+x+1} \ll (\ell - y)]_d + \left\lfloor \frac{\alpha_{i+x} \ll (\ell - y)}{d} \right\rfloor.$$

This proves the correctness.

□

3.3 Efficiency Analysis

We analyze the computational efficiency of each operation by counting the number of discrete bootstrappings. Note that discrete bootstrapping is usually > 100 times more costly than any other homomorphic operations (e.g. addition, multiplication), so it gives a good approximation of the computational complexity.

Theorem 4 (Reduction Complexity). *Let*

$$\text{ct} = (\text{ct}_0, \text{ct}_1, \dots, \text{ct}_{u-1}) \in (\mathcal{R}_Q^2)^u$$

be a vector of ciphertexts encrypting integer vectors in $[0, d^m - d^{m-1}]$, where $Q \gg d^m$. Then the number of discrete bootstrappings used to instantiate the Reduce operation is $\leq um - m + 1$.

Proof. We provide an algorithm with at most $um - m + 1$ bootstrappings. Recall that Reduce consists of u IntMod_d and $u - 1$ Carry_d . The main issue is that the definition of Carry_d includes homomorphic division by d , which consumes modulus. To handle this issue, we keep all the ct_i 's, ct_i' 's, ct_i'' 's at the bootstrapping (output) level via discrete bootstrapping. To do this, we instantiate Carry_d with the iterative bootstrapping in [KN25]. The key observation is that $\text{ct}_{i-1}'' + \text{ct}_i$ decrypts to an element $< d^m$, which can be proved by mathematical induction on i .

- Base case $i = 0$: ct_i decrypts to $\leq d^m - d^{m-1} < d^m$.
- Assume i and prove $i + 1$: Assume that $\text{ct}_{i-1}'' + \text{ct}_i$ decrypts to $< d^m$. Then by the definition of Carry_d , ct_i'' decrypts to $< d^{m-1}$. Thus, $\text{ct}_i'' + \text{ct}_{i+1}$ decrypts to

$$< d^{m-1} + d^m - d^{m-1} = d^m.$$

- By mathematical induction, we proved the desired property.

Therefore, Carry_d can be instantiated with IntBoot_d^{m-1} of [KN25, Algorithm 2] which requires $m - 1$ discrete bootstrappings. As a result, we instantiate Reduce with $u \cdot 1 + (u - 1) \cdot (m - 1) = um - m + 1$ discrete bootstrappings. $\square$

Corollary 1 (Addition/Subtraction Complexity). *The number of bootstrappings needed for addition/subtraction is $\leq 2u - 1$.*

Proof. It follows directly from the fact that $[0, d) + [0, d) = [0, 2d - 1)$ and $[0, d) - [0, d) = (-d, d)$ whose lengths are $\leq d^2 - d$. $\square$

Corollary 2 (Comparison Complexity). *The number of bootstrappings needed for comparison is $\leq 2u$.*

Proof. Recall that the comparison operation does exactly the same as the subtraction, except for the last carry step. Hence, we need $2u - 1 + 1 = 2u$ discrete bootstrappings. $\square$

Theorem 5 (Multiplication Complexity). *Suppose that $k \leq \ell \cdot 2^\ell$. The number of bootstrappings needed for multiplication is $\leq 3u - 2$.*

Proof. Recall that the Mult operation heavily relies on the Reduce operation. Hence, it suffices to check the size of the underlying message of

$$M_i = \sum_{j=0}^i \text{Mult}(\text{ct}_j, \text{ct}'_{i-j}).$$

As each ct_j and ct'_j decrypts to a ℓ -bit unsigned integer, we have that each coordinate of the decryption of M_i is less than or equal to

$$(2^\ell - 1) \cdot (2^\ell - 1) \cdot u = (d - 1)^2 \cdot u \leq (d - 1)^2 \cdot d < d^3 - d^2.$$

Hence, by Theorem 4, the number of bootstrappings needed is at most $3d - 2$. $\square$

Theorem 6 (Shift Complexity). *The number of bootstrappings needed for the right/left shift operation is $\leq u - x$.*

Proof. We first prove the left shift operation. Observe that we need to compute $\text{IntMod}_d(\text{ct}_i \ll y)$ for $0 \leq i < u - x$ and $\text{Carry}_d(\text{ct}_i \ll y)$ for $0 \leq i < u - x - 1$. We can compute $\text{IntMod}_d(\text{ct}_i \ll y)$ for $0 \leq i < u - x$ using $u - x$ discrete bootstrappings. The remaining Carry_d 's can be computed without introducing additional discrete bootstrappings. Hence $u - x$ in total.

The right shift operation works exactly the same except that the IntMod_d is needed for index $x \leq i < u$ and Carry_d is needed for index $x + 1 \leq i < u$. $\square$

We summarize the analyses in Table 3.

Table 3: Number of discrete bootstrapping used in each operation.

Operation Name	Number of Discrete Bootstrapping
Addition	$\leq 2u - 1$
Subtraction	
Multiplication	$\stackrel{8}{\leq} 3u - 2$
Comparison	$\leq 2u$
Right/Left Shift	$\leq u - x$

⁸Under the assumption that $k \leq \ell \cdot 2^\ell$.

Next, we discuss some optimization techniques that can further reduce the computational complexity. The first optimization is to reduce the number of discrete bootstrapping by lazily bootstrapping Carry_d . For instance, one may bootstrap for every γ (instead of 1) Carry_d when evaluating Reduce as in Theorem 4. Although it reduces the number of bootstrapping by a factor γ , it increases the modulus consumption by a factor γ , so one should carefully examine the efficiency to find a sweet spot.

Theorem 7 (Lazy Bootstrap). *Let $\text{ct} = (\text{ct}_0, \text{ct}_1, \dots, \text{ct}_{u-1}) \in (\mathcal{R}_Q^2)^u$ be a vector of ciphertexts encrypting integer vectors in $[0, d^m - d^{m-1}]$, where $Q \gg d^{\gamma m}$. Then there is an instantiation of Reduce that uses at most $m \cdot \lfloor (u-1)/\gamma \rfloor + u$ bootstrappings and $d^{m+\gamma-1}$ of modulus.*

Proof. We may instantiate exactly as in the proof of Theorem 4, except that we lazily bootstrap for Carry_d . To be explicit, we first obtain ct_i'' (via Carry_d operation with lazy bootstrapping) sequentially and compute ct_i' at the end (via u discrete bootstrappings). Here we may choose the ciphertext modulus of ct_i'' to be a factor d smaller than that of ct_{i-1}'' for $\gamma \nmid i$ and bootstrap at $\gamma \mid i$. As a result, the ciphertext modulus for ct_i'' becomes $Q/d^{\lfloor i \rfloor \gamma}$. Recall that the modulus consumption for the last bootstrapping (i.e. the iterative bootstrapping in [KN25]) is d^m and the number of bootstrapping is m . Hence the total modulus consumption is $d^m \cdot d^{\gamma-1} = d^{m+\gamma-1}$, and the total number of bootstrapping is

$$m \cdot (\# \text{ of bootstrap for } \text{Carry}_d) + u \cdot (\# \text{ of bootstrap for } \text{IntMod}_d) = m \cdot \lfloor (u-1)/\gamma \rfloor + u.$$

□

Another observation is that we may reduce lazily, especially in addition/subtraction. For instance, if we add three ciphertext vectors $\text{ct}, \text{ct}', \text{ct}'' \in (\mathcal{R}_Q^2)^u$, we may define the addition of them as

$$\text{Add}(\text{ct}, \text{ct}', \text{ct}'') = \text{Reduce}((\text{ct}_i + \text{ct}'_i + \text{ct}''_i)_{0 \leq i < u})$$

hence saving one Reduce operation. We formalize this observation in the following theorem.

Theorem 8 (Lazy Addition). *Let $\text{ct}^0, \text{ct}^1, \dots, \text{ct}^{v-1} \in (\mathcal{R}_Q^2)^u$ be vectors of ciphertexts encrypting unsigned k -bit integer vectors according to the encryption scheme in Section 3.1, where $v \leq d$. The addition of $\text{ct}^0, \text{ct}^1, \dots, \text{ct}^{v-1}$ can be instantiated as*

$$\text{Reduce} \left(\left(\sum_{j=0}^{v-1} \text{ct}_i^j \right)_{0 \leq i < u} \right)$$

with $\leq 2u - 1$ discrete bootstrappings.

Proof. By Theorem 4, we only need to check that for each i , $\sum_{j=0}^{v-1} \text{ct}_i^j$ decrypts to an integer vector whose entries are in $[0, d^2 - d]$. To check this, we use the fact that ct_i^j decrypts to an integer vector whose entries are in $[0, d - 1]$. Then, by summing up at most d such ciphertexts, we get the desired property. □

Thus, we may lazily reduce after adding $\leq d$ ciphertext vectors.

3.4 Implications

We discuss some direct applications of our method, namely fixed-point arithmetic and arbitrary function evaluation over unsigned integers.

3.4.1 Fixed Point Arithmetic

As our scheme supports both multiplication and shift operations, it immediately supports fixed-point multiplication, which can be written as a combination of multiplication of two m bit integers and right shift by m . However, we may do much better than this because the fixed-point multiplication can be regarded as computing the high part of the multiplication.

- **Extended Reduction:** Let $\text{ct} = (\text{ct}_0, \text{ct}_1, \dots, \text{ct}_{2u-1}) \in (\mathcal{R}_Q^2)^{2u}$ be a vector of discrete CKKS ciphertexts encrypting vectors in $\mathbb{Z}^{N/2}$. The (extended) modular reduction of ct is sequentially defined as

$$\text{ExtReduce}(\text{ct}) = (\text{ct}'_0, \text{ct}'_1, \dots, \text{ct}'_{2u-1})$$

where $\text{ct}'_0 = \text{IntMod}_d(\text{ct}_0)$, $\text{ct}''_0 = \text{Carry}_d(\text{ct}_0)$, and

$$\text{ct}'_i = \text{IntMod}_d(\text{ct}''_{i-1} + \text{ct}_i), \quad \text{ct}''_i = \text{Carry}_d(\text{ct}''_{i-1} + \text{ct}_i)$$

for each $1 \leq i < 2u$. The result of Reduce encrypts a message

$$\sum_{i=0}^{2u-1} [\text{Dcd} \circ \text{Dec}(\text{ct}_i)] \cdot d^i$$

modulo t^2 , i.e., reducing the ciphertext ct to the correct base- d representation.

- **Extended Multiplication:** Let $\text{ct} = (\text{ct}_i)_{0 \leq i < u}$, $\text{ct}' = (\text{ct}'_i)_{0 \leq i < u} \in (\mathcal{R}_Q^2)^u$ be vectors of ciphertexts each encrypting a k -bit unsigned integer according to the encryption scheme in Section 3.1. The extended multiplication of ct and ct' is defined as

$$\text{ExtMult}(\text{ct}, \text{ct}') = \text{ExtReduce} \left(\left(\sum_{j=\max(0, i-u+1)}^{\min(i, u-1)} \text{Mult}(\text{ct}_j, \text{ct}'_{i-j}) \right)_{0 \leq i < 2u} \right)$$

where Mult on the right-hand side refers to the usual CKKS multiplication. The output vector contains the element-wise product of two input integer vectors, without modular reduction by t .

One may simply take the last u entries of the result of the extended multiplication to instantiate a k -bit precision fixed-point multiplication. Recall that CKKS often struggles to satisfy the security notions like IND-CPA^D [LM21] or Threshold FHE [AJL⁺12, BGG⁺18] as they cannot separate noise from the message. Although our fixed-point arithmetic is much slower than the usual CKKS operations (as it involves bootstrapping), we may avoid huge flooding which greatly increases the parameters.

3.4.2 Arbitrary Function Evaluation

As our encoding stores a large integer after decomposing it into small pieces, we may directly benefit from the multi-precision arbitrary function evaluation in [AKP25, Section 5.3]. Since we maintain the digit-decomposed format, we do not need to perform digit extraction in [AKP25]. We restate (and slightly modify) the arbitrary function evaluation framework as follows.

Let $f : \mathbb{Z}_t \rightarrow \mathbb{C}$ be an arbitrary function, $\varphi : \mathbb{Z}_t \rightarrow \mathbb{Z}_d^u$ be the digit decomposition, and $\psi : \mathbb{Z}_d \rightarrow U_d = \{1, \omega, \dots, \omega^{d-1}\}$ be defined as $x \mapsto \omega^{2\pi i x}$ where ω is a complex d th root of unity. Let $p \in \mathbb{C}[x_0, \dots, x_{u-1}]$ be a multivariate polynomial that interpolates the function

Algorithm 4: Arbitrary Function Evaluation [AKP25]

Input : $\text{ct} = (\text{ct}_i)_{0 \leq i < u} \in (\mathcal{R}_Q^2)^u$ a ciphertext vector encrypting $\vec{z} \in \mathbb{Z}_t^{N/2}$.
 $f : \mathbb{Z}_t \rightarrow \mathbb{C}$ an arbitrary function.

Output : ct_{out} encrypting $f(\vec{z})$.

- 1 **for** $i \leftarrow 0$ **to** $u - 1$ **do**
- 2 | $\text{ct}'_i \leftarrow \text{EvalExp} \circ \text{CtS} \circ \text{ModRaise} \circ \text{StC}(\text{ct}_i);$
- 3 **end for**
- 4 $\text{ct}_{\text{out}} \leftarrow \text{MLUT}_f(\text{ct}'_0, \text{ct}'_1, \dots, \text{ct}'_{u-1});$
- 5 **return** ct_{out} .

$f \circ \varphi^{-1} \circ \psi^{-1} : U_d^u \rightarrow \mathbb{C}$. Then the homomorphic evaluation of p instantiates f (starting from complex roots of unity). We denote such an operation as MLUT_f . We illustrate the algorithm in Algorithm 4.

It is worth mentioning that such functionality cannot be supported if one handles large integers directly inside CKKS. One reason is that univariate interpolation consumes much more multiplicative depths than multivariate interpolation, and another reason is that large precision polynomial interpolation is numerically unstable.

4 Experiments

We provide proof-of-concept implementations for the algorithms described in the previous section. We developed our code upon the lattigo library [lat24]. The experiments are run single-threaded on Apple M4 Max with 128GB of RAM running macOS Sequoia 15.5. All of our FHE parameters satisfy ≈ 128 bit security according to [BTH22].

4.1 Description

We start with providing a parameter set used for our experiments, in Table 4. Here N denotes the RLWE ring dimension, $\log QP$ denotes the maximum RLWE modulus for switching keys, $(h, \tilde{h})$ denotes the Hamming weights of the dense and sparse secrets [BTH22], and $dnum$ denotes the gadget rank for the switching keys. The lower table describes the moduli chain, where $\log q_i$ denotes the ciphertext modulus and $\log p_j$ denotes the auxiliary modulus for key switching. **Base**, **StC**, **Mult**, **LUT**, **EvalExp**, and **CtS** denote the moduli reserved for the corresponding operations in the discrete bootstrapping framework [BKSS24]. When denoted as $X \times Y$, it refers to using Y many X -bit (NTT) moduli.

Table 4: Parameter set for the experiments.

$\log N$		$\log QP$	$(h, \tilde{h})$	$dnum$
15		768	(192, 32)	7

$\log q_i$						$\log p_j$
Base	StC	Mult	LUT	EvalExp	CtS	
36	28×2	36	36×5	36×8	32×2	36×3

Next, we elaborate on the scheme parameters of our experiments. We targeted 64-bit operations, decomposing it into 4-bit pieces, resulting in a total of 16 pieces. In other words, $k = 64$, $\ell = 4$, and $u = 16$. Note that $k \leq 2^\ell \cdot \ell$, which satisfies the condition for Theorem 5. For the homomorphic modular reduction instantiation, we followed [KN25], which relies on the discrete bootstrapping from [BKSS24]. In particular, we used the cleaning interpolation (via Hermite interpolation) so that our bootstrapping achieves both modulus raising and cleaning.

We implemented addition, subtraction, multiplication, comparison, and right shift operations relying on the algorithms in Section 3.2. In particular, we used the strategy to bootstrap every time we compute Carry_d , which was mainly used in the correctness proofs. This corresponds to the equality case in Table 3.

The experimental results are illustrated in Table 5. Here the error is measured by subtracting the decrypted result from the desired result. The precision denotes mean and minimum precision, which are computed as $-\log_2$ of the mean and maximum error, respectively. We observe that the total running time is roughly proportional to the number of bootstrappings, as expected.

Table 5: Experimental results for 64-bit homomorphic operations, based on the algorithms in Section 3.2 except shifting.

	# Bootstrap	Running time	Precision
Addition	31	96.8 sec	(16.5, 11.1)
Subtraction	31	99.1 sec	(16.5, 11.3)
Multiplication	46	145 sec	(14.2, 6.62)
Comparison	32	102 sec	(17.1, 12.9)

To check the cost of left and right shift operations, we checked if it is linear in $x = \lfloor m/d \rfloor$ as expected. We fixed $y = 1$ for the right shift and $y = 2$ for the left shift, and plotted the graph in Figure 2. Not only the linearity but also the fact that the cost of shift is independent of y was observed.

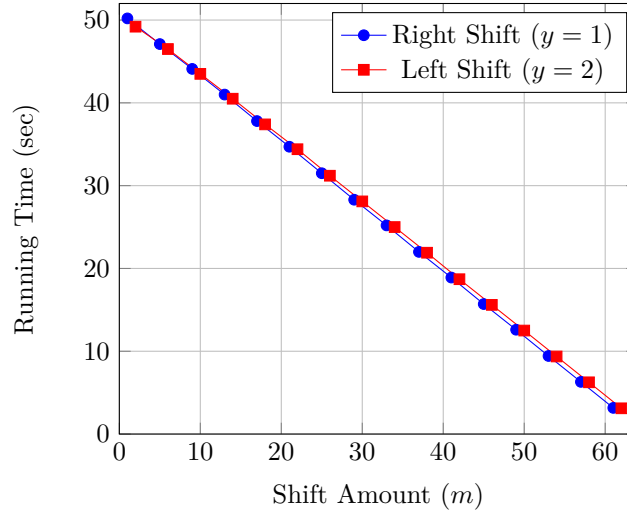


Figure 2: Running time of right/left shift operations.

4.2 Comparison with Prior Works

We mainly compare our algorithm with [Zam22] as it supports all the operations we discussed in Section 3.2. We ran their benchmark⁹ in the same environment as our experiments (e.g. same machine, single-thread, etc.). The figures are illustrated in Table 6. Our algorithm outperforms [Zam22] by 2-3 orders of magnitude, depending on the operations, in terms of throughput. Notably, our homomorphic 64-bit multiplication is

⁹We executed integer_bench from TFHE-rs [Zam22], commit hash 3129520.

more than 2800 times faster. To compare latency, our multiplication becomes favorable as soon as we have ≥ 6 parallelism.

Table 6: Comparison with TFHE-rs [Zam22] on 64-bit operations.

	Latency (sec)		Amortized time (ms)		
	[Zam22]	Ours	[Zam22]	Ours	
Addition	0.928	96.8	928	5.91	157x
Subtraction	0.930	99.1	930	6.05	154x
Multiplication	25.4	145	25400	8.85	2870x
Comparison	0.685	102	685	6.23	110x
Right Shift ¹⁰	0.467	50.2	467	3.06	153x
Left Shift	0.467	49.2	467	3.00	156x

Next, we dive deeper into homomorphic multiplications and see how the target modulus $t = 2^k$ affects the performance. We compare our multiplication with [Zam22] for $k = 4, 8, 16, 32, 64$, and check both latency and throughput. The figures can be seen in Table 7. Since our method is asymptotically almost linear in k but [Zam22] is not, the performance difference is widened for large k . Although we experimented $k \leq 64$ (due to the constraint $k \leq \ell \cdot 2^\ell$), our method can be easily extended by increasing ℓ . We expect that our method should perform even better for larger k .¹¹

Table 7: Comparison with TFHE-rs [Zam22] on multiplications of different sizes.

$k = \log_2 t$	Latency (sec)		Amortized time (ms)		
	[Zam22]	Ours	[Zam22]	Ours	
4	0.0872	3.16	87.2	0.193	452x
8	0.408	12.6	408	0.769	531x
16	1.62	31.2	1620	1.90	853x
32	6.42	69.6	6420	4.25	1510x
64	25.4	145	25400	8.85	2870x

We move on to other possible approaches based on SIMD schemes. Note that most works do not cover multiplication, comparison, and bootstrapping at the same time. For instance, [YZY⁺24] achieves good performance for both homomorphic multiplication and comparison, but the encoding formats for the two operations are different and do not have efficient conversions from one to another. In this regard, we focus on decomposition-based works that seem to naturally support all three operations we are interested in.

For arithmetic operations (addition and multiplication), we compare with [HZY⁺22], the state-of-the-art method for radix-2 homomorphic addition/multiplication. We borrowed the figures from [HZY⁺22, Table V and VIII], which uses a similar environment for experiments. As illustrated in Table 8, our addition is 4–8 times faster for computing 32/64-bit additions, and our multiplication is 240 times faster for evaluating 32-bit multiplication.

For homomorphic comparison, we compare with [TLW⁺21], which introduces the vector of field elements (VFE) encoding to efficiently evaluate large precision comparison. Although their latency is much better than ours, we outperform in terms of throughput due to the large parallelism of CKKS. The details are illustrated in Table 9.

¹⁰For right/left shift, we considered the smallest x which gives the largest running time for our method.

¹¹When $\ell > 4$, one may need to choose a larger N than Table 4. Roughly, this leads to 2 to 4 times larger bootstrapping latency. The noise can be controlled similarly. See [BKSS24] for details about scalability, as we directly rely on their discrete bootstrapping. As mentioned earlier, the number of bootstrappings for each homomorphic operation scales linearly with k , given a fixed ℓ .

¹²Borrowed from [TLW⁺21, Table 1]. Excluded Step (c) from the total time. Note that [IZ21] provides a more efficient solution ($\approx 2x$ faster), but they did not measure timings for modulo 2^{64} .

Table 8: Comparison with [HZY⁺22] on arithmetic operations.

		k	λ	amortized time (ms)
Addition	[HZY ⁺ 22]	32	80	24
		64		24
	Ours	32	≈ 128	2.90
		64		5.91
Multiplication	[HZY ⁺ 22]	32	80	1020
	Ours	32	≈ 128	4.25

Table 9: Homomorphic comparison over $\mathbb{Z}_{2^{64}}$, compared with [TLW⁺21].

	λ	# slots	latency (sec)	amortized time (ms)
[TLW ⁺ 21] ¹²	> 80	256	20.2	78.8
		128	20.5	160
		16	4.75	297
Ours	≈ 128	16384	102	6.23

One could also consider comparing our homomorphic comparison with the usual CKKS comparison [CKK20, LLKN22]. In [LLKN22, Table 7], they show ≈ 30 seconds for 20 bit precision homomorphic comparison, but do not show higher precision implementations. Asymptotically, the approximation based methods achieve $O(\alpha)$ computational complexity for a precision parameter α [CKK20, Table 2]. For $\alpha = 64$ bit precision, their performance can be roughly estimated as $30 \times (64/20) = 96$ seconds. Note that the asymptotic complexity only counts the number of homomorphic operations. For increased α , one needs to choose a higher scaling factor Δ , leading to higher modulus q and higher ring degree N , significantly increasing the running time.¹³ In addition, their methods are not directly compatible with our approach, as our encoding stores 64 bit integers in many low-precision discrete CKKS ciphertexts.

Memory. The public evaluation keys used in our method is exactly the same as the bootstrapping keys for the base discrete bootstrapping [BKSS24]. For instance, each evaluation key of the parameter set in Table 4 is of size 77 MiB, and there can be (roughly) 8 to 50 of them depending on the bootstrapping optimizations [HS18], leading to 0.6 to 4 gigabytes. The ciphertext size depends on the plaintext modulus $t = 2^k$, in the sense that there are k/ℓ CKKS ciphertexts per each large precision ciphertext. Concretely, a single $k = 64$ bit ciphertext (with $\ell = 4$) at the highest level 4 is $2 \text{ MiB} \times (k/\ell) = 32 \text{ MiB}$, and it becomes 64 MiB when $k = 128$.

When compared to methods based on BGV-like schemes, both the evaluation key and ciphertext sizes are similar. For example, for 32 bit multiplication, [HZY⁺22, Table 8] used 0.680 to 28.0 GB of memory. On the other hand, CGGI/DM based approaches (e.g. [Zam22]) use far less memory. Their functional/programmable bootstrapping often needs bootstrapping key size of less than a hundred megabytes [LMSS23, Table 6], which is an order of magnitude smaller than (discrete) CKKS.

4.3 Homomorphic EdDSA Signing

In [BK25], their main application is homomorphic signing, which provides one-round threshold/blind signatures for signature schemes like EdDSA [BDL⁺12]. They propose

¹³To estimate the impact, one can roughly choose $64/20 = 3.2$ times larger $\log_2(\Delta)$, leading to 3.2 times larger $\log_2(q)$ and N , and the runtime roughly becomes $3.2^2 \times 96 = 983$ seconds.

a dedicated homomorphic encryption scheme that supports homomorphic modular multiplication by an arbitrary prescribed modulus. For EdDSA, a popular choice is using a curve 25519, which relies on arithmetic operations modulo $p = 2^{255} - 19$. We observe that for such a Solinas prime, our homomorphic integer computer can give better performance in terms of throughput. We instantiated the homomorphic modular multiplication using 256 bit extended multiplication, addition, and comparison. Our experimental result shows a factor of ≈ 18 better throughput compared to [BK25], as illustrated in Table 11. Our approach can be preferable when there are sufficiently many signings in parallel.¹⁴

Table 10: Parameter set for the homomorphic signing experiment. We used $d = 2^8$ to instantiate $k = 512$ bit arithmetic.

$\log N$		$\log QP$	(h, h)	$dnum$
16		1470	(192, 32)	5

$\log q_i$						$\log p_j$
Base	StC	Mult	LUT	EvalExp	CtS	
50	43×3	50	50×9	50×8	47×3	
						50×5

Table 11: Comparison with [BK25] on modulo $p = 2^{255} - 19$ multiplications.

	# slots	latency (sec)	amortized time (ms)
[BK25]	32	150	4690
Ours	16384	4260	260

5 Conclusion

In this work, we constructed a homomorphic encryption scheme that supports a wide range of standard unsigned integer computations, such as addition, multiplication, comparison, and bit shift. The biggest strength of our method is throughput, which shows several orders of magnitude better performance than that of TFHE-rs.

We view our framework as a good baseline for SIMD homomorphic computations in general. The existing SIMD FHE schemes primarily support only addition and multiplication, often requiring dedicated approaches for general computations. On the other hand, the CGGI/DM family has built a rich community on top of the functional/programmable bootstrapping paradigm. We believe that our work helps emulate their success in the corresponding SIMD FHE world.

Acknowledgement

The work was supported by Stanford Graduate Fellowship, NSF, DARPA, and the Simons Foundation. Opinions, findings, and conclusions or recommendations expressed in this material are those of the authors and do not necessarily reflect the views of DARPA. We thank Professor Dan Boneh for helpful comments and suggestions, especially on applications and experiments.

¹⁴Note that our method is exact modular reduction, whereas [BK25] provides approximate modular reduction. Our method is more compatible with applying other operations afterwards.

References

- [AJL⁺12] Gilad Asharov, Abhishek Jain, Adriana López-Alt, Eran Tromer, Vinod Vaikuntanathan, and Daniel Wichs. Multiparty computation with low communication, computation and interaction via threshold FHE. In David Pointcheval and Thomas Johansson, editors, *Advances in Cryptology – EUROCRYPT 2012*, volume 7237 of *Lecture Notes in Computer Science*, pages 483–501, Cambridge, UK, April 15–19, 2012. Springer Berlin Heidelberg, Germany.
- [AKP25] Andreea Alexandru, Andrey Kim, and Yuriy Polyakov. General functional bootstrapping using CKKS. In *CRYPTO*. Springer-Verlag, 2025.
- [BCC⁺22] Youngjin Bae, Jung Hee Cheon, Wonhee Cho, Jaehyung Kim, and Taekyung Kim. META-BTS: Bootstrapping precision beyond the limit. In Heng Yin, Angelos Stavrou, Cas Cremers, and Elaine Shi, editors, *ACM CCS 2022: 29th Conference on Computer and Communications Security*, pages 223–234, Los Angeles, CA, USA, November 7–11, 2022. ACM Press.
- [BCK⁺23] Youngjin Bae, Jung Hee Cheon, Jaehyung Kim, Jai Hyun Park, and Damien Stehlé. HERMES: Efficient ring packing using MLWE ciphertexts and application to transciphering. In Helena Handschuh and Anna Lysyanskaya, editors, *Advances in Cryptology – CRYPTO 2023, Part IV*, volume 14084 of *Lecture Notes in Computer Science*, pages 37–69, Santa Barbara, CA, USA, August 20–24, 2023. Springer, Cham, Switzerland.
- [BCKS24] Youngjin Bae, Jung Hee Cheon, Jaehyung Kim, and Damien Stehlé. Bootstrapping bits with CKKS. In Marc Joye and Gregor Leander, editors, *Advances in Cryptology – EUROCRYPT 2024, Part II*, volume 14652 of *Lecture Notes in Computer Science*, pages 94–123, Zurich, Switzerland, May 26–30, 2024. Springer, Cham, Switzerland.
- [BDL⁺12] Daniel J. Bernstein, Niels Duif, Tanja Lange, Peter Schwabe, and Bo-Yin Yang. High-speed high-security signatures. *Journal of Cryptographic Engineering*, 2(2):77–89, September 2012.
- [BGG⁺18] Dan Boneh, Rosario Gennaro, Steven Goldfeder, Aayush Jain, Sam Kim, Peter M. R. Rasmussen, and Amit Sahai. Threshold cryptosystems from threshold fully homomorphic encryption. In Hovav Shacham and Alexandra Boldyreva, editors, *Advances in Cryptology – CRYPTO 2018, Part I*, volume 10991 of *Lecture Notes in Computer Science*, pages 565–596, Santa Barbara, CA, USA, August 19–23, 2018. Springer, Cham, Switzerland.
- [BGGJ20] Christina Boura, Nicolas Gama, Mariya Georgieva, and Dimitar Jetchev. Chimera: Combining ring-lwe-based fully homomorphic encryption schemes. *Journal of Mathematical Cryptology*, 14(1):316–338, 2020.
- [BGV12] Zvika Brakerski, Craig Gentry, and Vinod Vaikuntanathan. (Leveled) fully homomorphic encryption without bootstrapping. In Shafi Goldwasser, editor, *ITCS 2012: 3rd Innovations in Theoretical Computer Science*, pages 309–325, Cambridge, MA, USA, January 8–10, 2012. Association for Computing Machinery.
- [BK25] Dan Boneh and Jaehyung Kim. Homomorphic encryption for large integers from nested residue number systems. In *CRYPTO*. Springer-Verlag, 2025.

- [BKSS24] Youngjin Bae, Jaehyung Kim, Damien Stehlé, and Elias Suvanto. Bootstrapping small integers with CKKS. In Kai-Min Chung and Yu Sasaki, editors, *Advances in Cryptology – ASIACRYPT 2024, Part I*, volume 15484 of *Lecture Notes in Computer Science*, pages 330–360, Kolkata, India, December 9–13, 2024. Springer, Singapore, Singapore.
- [Bra12] Zvika Brakerski. Fully homomorphic encryption without modulus switching from classical GapSVP. In Reihaneh Safavi-Naini and Ran Canetti, editors, *Advances in Cryptology – CRYPTO 2012*, volume 7417 of *Lecture Notes in Computer Science*, pages 868–886, Santa Barbara, CA, USA, August 19–23, 2012. Springer Berlin Heidelberg, Germany.
- [BTH22] Jean-Philippe Bossuat, Juan Ramón Troncoso-Pastoriza, and Jean-Pierre Hubaux. Bootstrapping for approximate homomorphic encryption with negligible failure-probability by using sparse-secret encapsulation. In Giuseppe Ateniese and Daniele Venturi, editors, *ACNS 2022: 20th International Conference on Applied Cryptography and Network Security*, volume 13269 of *Lecture Notes in Computer Science*, pages 521–541, Rome, Italy, June 20–23, 2022. Springer, Cham, Switzerland.
- [CGGI16] Ilaria Chillotti, Nicolas Gama, Mariya Georgieva, and Malika Izabachène. Faster fully homomorphic encryption: Bootstrapping in less than 0.1 seconds. In Jung Hee Cheon and Tsuyoshi Takagi, editors, *Advances in Cryptology – ASIACRYPT 2016, Part I*, volume 10031 of *Lecture Notes in Computer Science*, pages 3–33, Hanoi, Vietnam, December 4–8, 2016. Springer Berlin Heidelberg, Germany.
- [CHK⁺18] Jung Hee Cheon, Kyoohyung Han, Andrey Kim, Miran Kim, and Yongsoo Song. Bootstrapping for approximate homomorphic encryption. In Jesper Buus Nielsen and Vincent Rijmen, editors, *Advances in Cryptology – EUROCRYPT 2018, Part I*, volume 10820 of *Lecture Notes in Computer Science*, pages 360–384, Tel Aviv, Israel, April 29 – May 3, 2018. Springer, Cham, Switzerland.
- [CHM⁺25] Hyunho Cha, Intak Hwang, Seonhong Min, Jinyeong Seo, and Yongsoo Song. MatriGear: Accelerating Authenticated Matrix Triple Generation with Scalable Prime Fields via Optimized HE Packing . In *2025 IEEE Symposium on Security and Privacy (SP)*, pages 2453–2471, Los Alamitos, CA, USA, May 2025. IEEE Computer Society.
- [CJP21] Ilaria Chillotti, Marc Joye, and Pascal Paillier. Programmable bootstrapping enables efficient homomorphic inference of deep neural networks. In Shlomi Dolev, Oded Margalit, Benny Pinkas, and Alexander Schwarzmänn, editors, *Cyber Security Cryptography and Machine Learning*, pages 1–19, Cham, 2021. Springer International Publishing.
- [CKK16] Jung Hee Cheon, Miran Kim, and Myungsun Kim. Optimized search-and-compute circuits and their application to query evaluation on encrypted data. *IEEE Transactions on Information Forensics and Security*, 11(1):188–199, 2016.
- [CKK20] Jung Hee Cheon, Dongwoo Kim, and Duhyeong Kim. Efficient homomorphic comparison methods with optimal complexity. In Shiho Moriai and Huaxiong Wang, editors, *Advances in Cryptology – ASIACRYPT 2020, Part II*, volume 12492 of *Lecture Notes in Computer Science*, pages 221–256, Daejeon, South Korea, December 7–11, 2020. Springer, Cham, Switzerland.

- [CKKL24] Heewon Chung, Hyojun Kim, Young-Sik Kim, and Yongwoo Lee. Amortized large look-up table evaluation with multivariate polynomials for homomorphic encryption. *Cryptology ePrint Archive*, Report 2024/274, 2024.
- [CKKS17] Jung Hee Cheon, Andrey Kim, Miran Kim, and Yong Soo Song. Homomorphic encryption for arithmetic of approximate numbers. In Tsuyoshi Takagi and Thomas Peyrin, editors, *Advances in Cryptology – ASIACRYPT 2017, Part I*, volume 10624 of *Lecture Notes in Computer Science*, pages 409–437, Hong Kong, China, December 3–7, 2017. Springer, Cham, Switzerland.
- [DM15] Léo Ducas and Daniele Micciancio. FHEW: Bootstrapping homomorphic encryption in less than a second. In Elisabeth Oswald and Marc Fischlin, editors, *Advances in Cryptology – EUROCRYPT 2015, Part I*, volume 9056 of *Lecture Notes in Computer Science*, pages 617–640, Sofia, Bulgaria, April 26–30, 2015. Springer Berlin Heidelberg, Germany.
- [DMPS24] Nir Drucker, Guy Moshkovich, Tomer Pelleg, and Hayim Shaul. BLEACH: Cleaning errors in discrete computations over CKKS. *Journal of Cryptology*, 37(1):3, January 2024.
- [FV12] Junfeng Fan and Frederik Vercauteren. Somewhat practical fully homomorphic encryption. *Cryptology ePrint Archive*, Report 2012/144, 2012.
- [Gen09] Craig Gentry. Fully homomorphic encryption using ideal lattices. In Michael Mitzenmacher, editor, *41st Annual ACM Symposium on Theory of Computing*, pages 169–178, Bethesda, MD, USA, May 31 – June 2, 2009. ACM Press.
- [GIKV23] Robin Geelen, Ilia Iliashenko, Jiayi Kang, and Frederik Vercauteren. On polynomial functions modulo p^e and faster bootstrapping for homomorphic encryption. In Carmit Hazay and Martijn Stam, editors, *Advances in Cryptology – EUROCRYPT 2023, Part III*, volume 14006 of *Lecture Notes in Computer Science*, pages 257–286, Lyon, France, April 23–27, 2023. Springer, Cham, Switzerland.
- [GV23] Robin Geelen and Frederik Vercauteren. Bootstrapping for BGV and BFV revisited. *J. Cryptol.*, 36(2), March 2023.
- [GV25] Robin Geelen and Frederik Vercauteren. Fully homomorphic encryption for cyclotomic prime moduli. In Serge Fehr and Pierre-Alain Fouque, editors, *Advances in Cryptology – EUROCRYPT 2025, Part III*, volume 15603 of *Lecture Notes in Computer Science*, pages 366–397, Madrid, Spain, May 4–8, 2025. Springer, Cham, Switzerland.
- [HS18] Shai Halevi and Victor Shoup. Faster homomorphic linear transformations in HELib. In Hovav Shacham and Alexandra Boldyreva, editors, *Advances in Cryptology – CRYPTO 2018, Part I*, volume 10991 of *Lecture Notes in Computer Science*, pages 93–120, Santa Barbara, CA, USA, August 19–23, 2018. Springer, Cham, Switzerland.
- [HZY⁺22] Zongsheng Hou, Neng Zhang, Bohan Yang, Hanning Wang, Min Zhu, Shouyi Yin, Shaojun Wei, and Leibo Liu. Efficient fhe radix-2 arithmetic operations based on redundant encoding. *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems*, 41(7):2024–2037, 2022.
- [IZ21] Ilia Iliashenko and Vincent Zucca. Faster homomorphic comparison operations for BGV and BFV. *Proceedings on Privacy Enhancing Technologies*, 2021(3):246–264, July 2021.

- [KN25] Jaehyung Kim and Taeyeon Noh. Modular reduction in CKKS. *IACR Communications in Cryptology*, 2(2), 2025.
- [KS23] Kamil Klucznik and Leonard Schild. FDFB: Full domain functional bootstrapping towards practical fully homomorphic encryption. *IACR Transactions on Cryptographic Hardware and Embedded Systems*, 2023(1):501–537, 2023.
- [KSS24] Jaehyung Kim, Jinyeong Seo, and Yongsoo Song. Simpler and faster BFV bootstrapping for arbitrary plaintext modulus from CKKS. In Bo Luo, Xiaojing Liao, Jun Xu, Engin Kirda, and David Lie, editors, *ACM CCS 2024: 31st Conference on Computer and Communications Security*, pages 2535–2546, Salt Lake City, UT, USA, October 14–18, 2024. ACM Press.
- [lat24] Lattigo v6. Online: <https://github.com/tuneinsight/lattigo>, 2024. EPFL-LDS, Tune Insight SA.
- [LLKN22] Eunsang Lee, Joon-Woo Lee, Young-Sik Kim, and Jong-Seon No. Optimization of homomorphic comparison algorithm on RNS-CKKS scheme. *IEEE Access*, 10:26163–26176, 2022.
- [LM21] Baiyu Li and Daniele Micciancio. On the security of homomorphic encryption on approximate numbers. In Anne Canteaut and François-Xavier Standaert, editors, *Advances in Cryptology – EUROCRYPT 2021, Part I*, volume 12696 of *Lecture Notes in Computer Science*, pages 648–677, Zagreb, Croatia, October 17–21, 2021. Springer, Cham, Switzerland.
- [LMSS23] Changmin Lee, Seonhong Min, Jinyeong Seo, and Yongsoo Song. Faster ttfhe bootstrapping with block binary keys. In *Proceedings of the 2023 ACM Asia Conference on Computer and Communications Security*, ASIA CCS ’23, page 2–13, New York, NY, USA, 2023. Association for Computing Machinery.
- [LW23] Zeyu Liu and Yunhao Wang. Amortized functional bootstrapping in less than 7 ms, with $\tilde{O}(1)$ polynomial multiplications. In Jian Guo and Ron Steinfeld, editors, *Advances in Cryptology – ASIACRYPT 2023, Part VI*, volume 14443 of *Lecture Notes in Computer Science*, pages 101–132, Guangzhou, China, December 4–8, 2023. Springer, Singapore, Singapore.
- [MHW24] Shihe Ma, Tairong Huang, Anyu Wang, and Xiaoyun Wang. Accelerating BGV bootstrapping for large p using null polynomials over $\mathbb{Z}_{p^e}$. In Marc Joye and Gregor Leander, editors, *Advances in Cryptology – EUROCRYPT 2024, Part II*, volume 14652 of *Lecture Notes in Computer Science*, pages 403–432, Zurich, Switzerland, May 26–30, 2024. Springer, Cham, Switzerland.
- [MS18] Daniele Micciancio and Jessica Sorrell. Ring Packing and Amortized FHEW Bootstrapping. In Ioannis Chatzigiannakis, Christos Kaklamanis, Daniel Marx, and Donald Sannella, editors, *45th International Colloquium on Automata, Languages, and Programming (ICALP 2018)*, volume 107 of *Leibniz International Proceedings in Informatics (LIPIcs)*, pages 100:1–100:14, Dagstuhl, Germany, 2018. Schloss Dagstuhl – Leibniz-Zentrum für Informatik.
- [NHY+25] Chao Niu, Zhicong Huang, Zhaomin Yang, Yi Chen, Liang Kong, Cheng Hong, and Tao Wei. XBOOT: Free-XOR gates for CKKS with applications to transciphering. Cryptology ePrint Archive, Report 2025/074, 2025.

- [PMH⁺25] Thales B. Paiva, Gabrielle De Micheli, Syed Mahbub Hafiz, Marcos A. Simplicio Jr., and Bahattin Yildiz. Faster amortized bootstrapping using the incomplete NTT for free. *Cryptology ePrint Archive*, Paper 2025/696, 2025.
- [QZL⁺19] Qiao Qin, Neng Zhang, Leibo Liu, Shouyi Yin, and Shaojun Wei. Addition circuit optimization using carry-lookahead and simd for homomorphic encryption. In *2019 IEEE International Conference on Electron Devices and Solid-State Circuits (EDSSC)*, pages 1–3, 2019.
- [TLW⁺21] Benjamin Hong Meng Tan, Hyung Tae Lee, Huaxiong Wang, Shuqin Ren, and Khin Mi Mi Aung. Efficient private comparison queries over encrypted databases using fully homomorphic encryption with finite fields. *IEEE Transactions on Dependable and Secure Computing*, 18(6):2861–2874, 2021.
- [XCWF16] Chen Xu, Jingwei Chen, Wenyuan Wu, and Yong Feng. Homomorphically encrypted arithmetic operations over the integer ring. In Feng Bao, Liqun Chen, Robert H. Deng, and Guojun Wang, editors, *Information Security Practice and Experience*, pages 167–181, Cham, 2016. Springer International Publishing.
- [Zam22] Zama. TFHE-rs: A Pure Rust Implementation of the TFHE Scheme for Boolean and Integer Arithmetics Over Encrypted Data, 2022. <https://github.com/zama-ai/tfhe-rs>.
- [ZQH⁺21] Neng Zhang, Qiao Qin, Zongsheng Hou, Bohan Yang, Shouyi Yin, Shaojun Wei, and Leibo Liu. Efficient comparison and addition for FHE with weighted computational complexity model. *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems*, 40(9):1896–1908, 2021.
- [ZYZ⁺24] Fahong Zhang, Chen Yang, Rui Zong, Xinran Zheng, Jianfei Wang, and Yishuo Meng. An efficient and scalable fhe-based pdq scheme: Utilizing fft to design a low multiplication depth large-integer comparison algorithm. *IEEE Transactions on Information Forensics and Security*, 19:2258–2272, 2024.